Data Science

Model Generalizability

GENERAL ASSEMBLY

Lesson roadmap

- Welcome + Warm-Up
- What Is Generalizability?
- Train-Test Split
- k-Folds Cross Validation
- Practical Considerations
- Bring It Home

Learning objectives

1. Describe the principles of Generalizability.
2. Compare Bias and Variance and describe the tradeoff between when selecting a model.
3. Define and apply the Train-Test Split method using `scikit-learn`.
4. Explore and apply the k-Folds Cross Validation method in `scikit-learn`.

Warm-Up

© 2024 General Assembly

How long will you live?

Let's say we want to estimate how long you will live.

We have access to a database that contains lots of information on people's lives and how they lived. We want to use it to predict your chance of making it to 80 years old.

We could look at 2 features:

1. **Age**: everybody who reached the same age as you are now
2. **Background**: people with the same socioeconomic status

This gives us 10,000 people to compare with. 8,000 of them reached 80 years old, so we could say you have an **80% chance (probability) of reaching 80**.

 © 2025 General Assembly

Adapted from The Art of Statistics, David Spiegelhalter

How long will you live? (cont.)

But how accurate is that prediction? It is only based on 2 features, and misses many other important characteristics that could improve our prediction, for example:

- Are you a smoker?
- How much alcohol do you drink?
- What is your weight?
- What is your blood pressure?
- What is your cholesterol?
- How much exercise do you take?
- etc...

Including all of these can give us much more accurate comparators, in this case 2 people. One of them died before they were 80, so now our prediction is that you have a **50% chance of reaching 80**.

Adapted from The Art of Statistics, David Spiegelhalter

Comparing Two Models

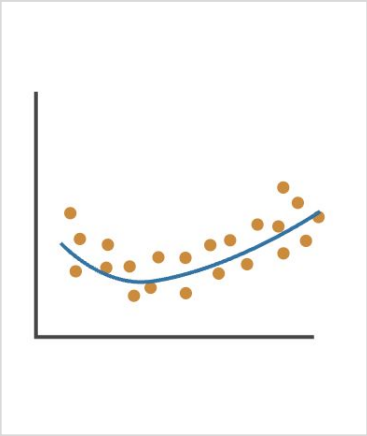
- ➔ Which of the 2 models is better?
- ➔ And why?

 © 2025 General Assembly

What Is Generalizability?

Goal of Model Building

The goal of most models we build is to learn from a limited amount of data (**training data**) and **apply it to new data** to make accurate predictions. This is known as a model that generalizes well i.e. it is **generalizable**. While we are building out our models we are aiming find a balance between being over and underfit.

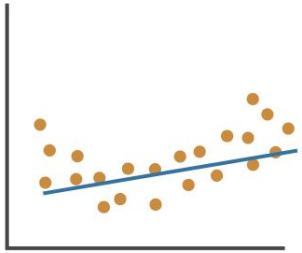


Source

Model Fit

Underfit

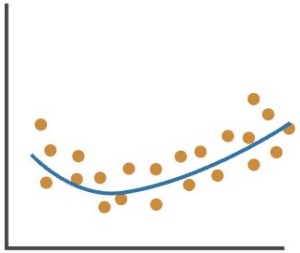
Doesn't capture the shape of the data (curve/ parabolic shape)



Training Data: High Error
New Data: High Error

Balanced Model

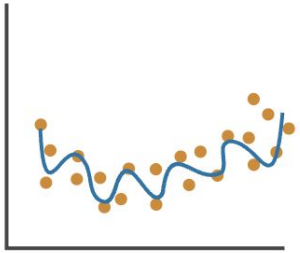
This model is a good balance that captures the training data well and will work well for new data



Training Data: Medium Error
New Data: Low Error

Overfit

Memorizes each data point and won't predict new values accurately



Training Data: Low Error
New Data: High Error

Source

Error is Error, Right?

Don’t confuse error with “residual”. There are actually three (3!) types of error. You can think of a residual as the sum of all 3 types.

 © 2025 General Assembly

The Three Types of Error

BIAS	This is the error attributable to our model's inability to adequately predict a value. You can think of this plainly as "model badness".
VARIANCE	Variance describes the spread of your data. Errors attributable variance may result in your model being too specific because it hasn't learned from a broad enough set of examples. That is, if you split your data in half and fit the same type of model on each half, variance is why you'll get slightly different models.
IRREDUCIBLE ERROR	This is error attributable to the random nature of the universe. This can never be reduced or eliminated by statistical techniques. It is sometimes called **noise**.

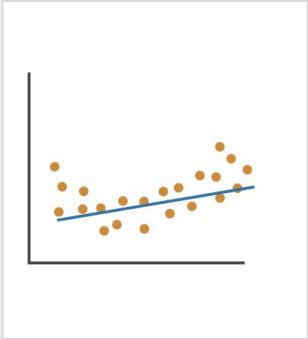
 © 2025 General Assembly

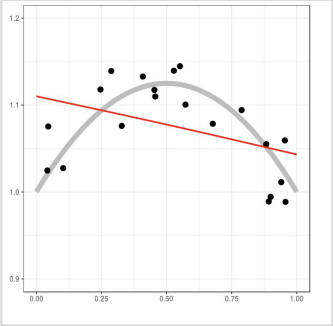
Bias

Our model (the blue straight line in the top graph, and the red straight line in the bottom graph) does not adequately capture the complexity of nature.

The model does a poor job of modeling nature. There are forces at play we're not accounting for.

This model is said to have **high bias** and is thus **underfit**.

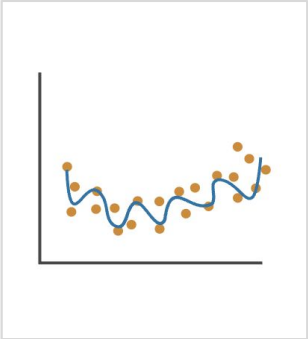


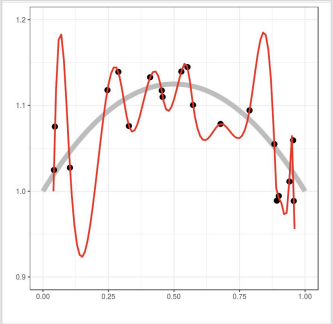


Variance

In this case, the model (blue line in the top graph and red line in the bottom one) is far too complicated for nature. It's not going to generalize to new data properly.

This model is said to have **high variance** and is thus **overfit**.





Another way to think about it

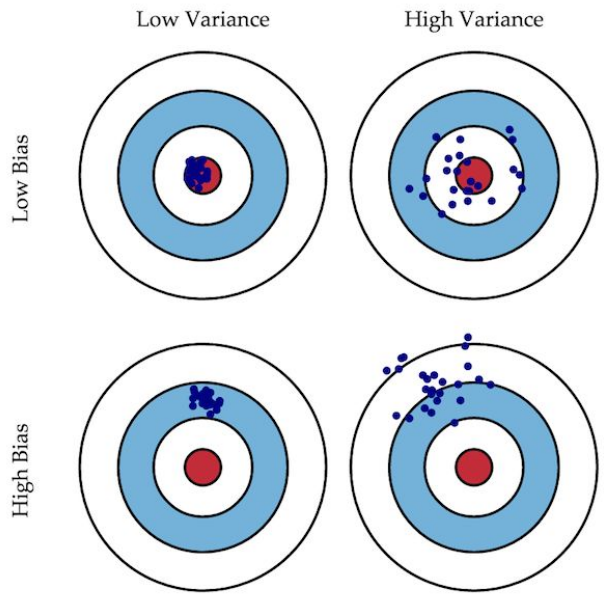


Fig. 1 Graphical illustration of bias and variance.

Source

Comparison of Model Fit

UNDERFIT MODELS	OVERFIT MODELS
Poor fit to training data Performs on new data just as well as old data (that is, poorly) **Model is too simple** to model real life in this case.	Very good fit to training data Does not generalize to new data **Model is too complex** Real life turned out to be simpler in this case.
Model with 2 features	*Model with 2 similar people*

 © 2025 General Assembly

What is model complexity?

Model **Complexity** is what it sounds like: how complicated is it to describe our model?

Generally, more complicated models have a finer control on the model's prediction, which can actually be a bad thing.

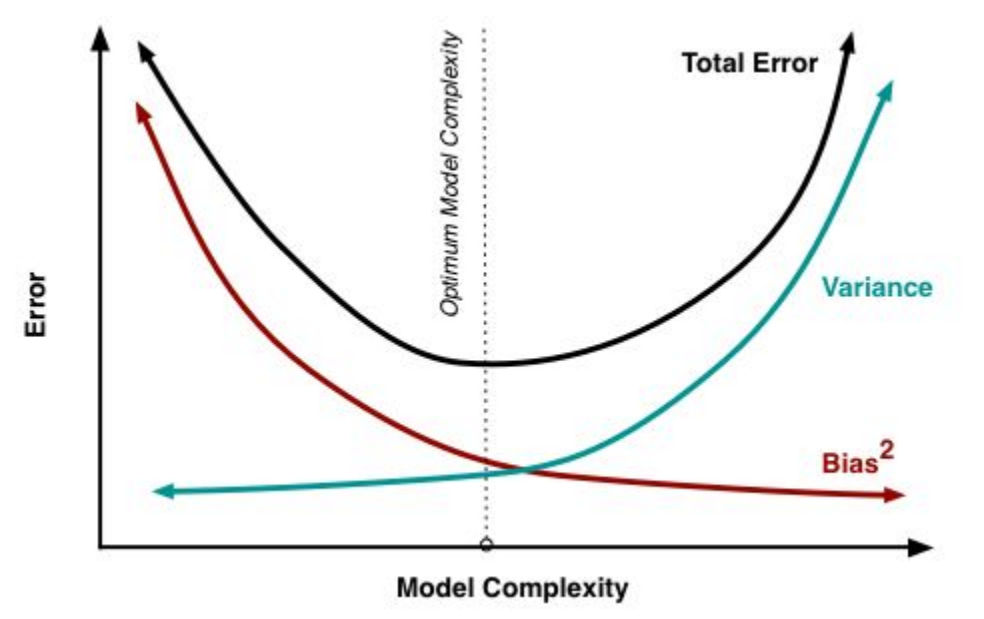
Here are a few things that control model complexity:

- Number of variables in the model.
- "Wiggliness" of variables included (polynomial fits: x, x^2, x^3, etc.).
- Level of regularization (a topic for future lessons).
- Model architecture (e.g., neural nets are more complicated than OLS).

 © 2025 General Assembly

The Whole Point

Our goal is to balance this **bias-variance tradeoff**! But how?!



Source

Measuring the Bias Variance Tradeoff

It turns out, bias and variance are theoretical quantities and cannot be calculated directly from the data. We can **estimate** them with **holdout techniques**, however!

We'll start with the most famous holdout technique, the **train-test split**.

 © 2025 General Assembly

Train-Test Split

Which Metrics to Choose?

To estimate the bias-variance tradeoff, we'll need a metric to use as a benchmark.

For regression problems, it's traditional to use the **MSE**.

 © 2025 General Assembly

The Train-Test Split

For this technique, we will divide our data into two pieces: the **training set** and the **testing set**.

 © 2025 General Assembly

The Train-Test Split

Training MSE:
150

A model is fit using only the **training data** and a **training metric** is computed.

 © 2025 General Assembly

The Train-Test Split

Training MSE:
150

Testing MSE:
???

A model is fit using only the training data and a **training metric** is computed. Often it's the MSE.

Then, using the same model, we compute a **testing metric** to see how the model performs on **new data**.

 © 2025 General Assembly

The Train-Test Split

Training MSE:
150

Testing MSE:
250

If the testing metric is much higher (worse) than the training metric, that indicates our model is **overfit**, since it has not generalized to new data very well (**low generalizability**).

 © 2025 General Assembly

The Train-Test Split

Training MSE:
150

Testing MSE:
160

If the testing metric is “close enough” to the training metric, then our model is likely not overfit.

 © 2025 General Assembly

What is the best train-test ratio?

How much data should be held out for our testing data?

There's actually no hard rule for this, but there's some good advice out there.

Generally, **you want two things to be true**:

1. You want as much data as possible for model fitting (training data).
2. You want enough testing data to adequately reflect all the types of data you might encounter in the real world.

 © 2025 General Assembly

Train-Test Split Rule-of-Thumb

If you have **small data**, you'll want a **higher proportion of data in the test set**. For example, if you only have 200 samples, you *might* opt for a 50/50 split! This is because you need a representative test set.

If you have **large data**, you'll want a **much larger proportion of data in the training data**.

For example, if you have 100M rows, you might opt for a 99/1 split! This is because you want as many training samples as possible, and 1M is more than sufficient for a representative test set.

In general, however, the default 75/25 or 70/30 split is usually fine.

 © 2025 General Assembly

Describe this model to me...

$$y \sim x_1 + x_2 + x_1x_2 + x_3 + x_3^2 + x_4 + x_5$$

Train RMSE = 12.3

Test RMSE = 658.2

 © 2025 General Assembly

...and this one...

$y \sim x_1 + x_2 + x_1x_2 + x_3 + x_3^2 + x_4 + x_5$

Train RMSE = 12.3

Test RMSE = 658.2

$y \sim x_1 + x_2$

Train RMSE = 233.9

Test RMSE = 252.1

 © 2025 General Assembly

How about this one?

$y \sim x_1 + x_2 + x_1x_2 + x_3 + x_3^2 + x_4 + x_5$

Train RMSE = 12.3

Test RMSE = 658.2

$y \sim x_1 + x_2$

Train RMSE = 233.9

Test RMSE = 252.1

$y \sim x_3 + x_4$

Train RMSE = 132.7

Test RMSE = 200.5

 © 2025 General Assembly

Train-Test Split

Let's see it for ourselves!

- ☐ Open **02-train-test-split-cross-validation.ipynb**

k-Folds Cross Validation

k-Fold Cross-Validation

There's another holdout technique that might be preferable in some situations: the ***k*-fold cross-validation** technique.

 © 2025 General Assembly

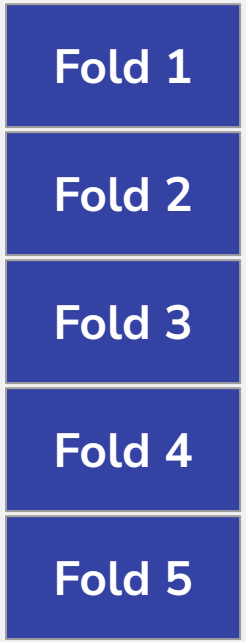
k-Fold Cross-Validation

To begin, select a number *k*, and split your data into ***k*-folds**.

This is not a tuning parameter, but *k* should be large enough so that averaging *k* values is meaningful, but also small enough so that splitting your data into *k* chunks doesn't make the folds too small.

 © 2025 General Assembly

k-Fold Cross-Validation



We'll use $k = 5$ folds. Common choices are usually 5 or 10.

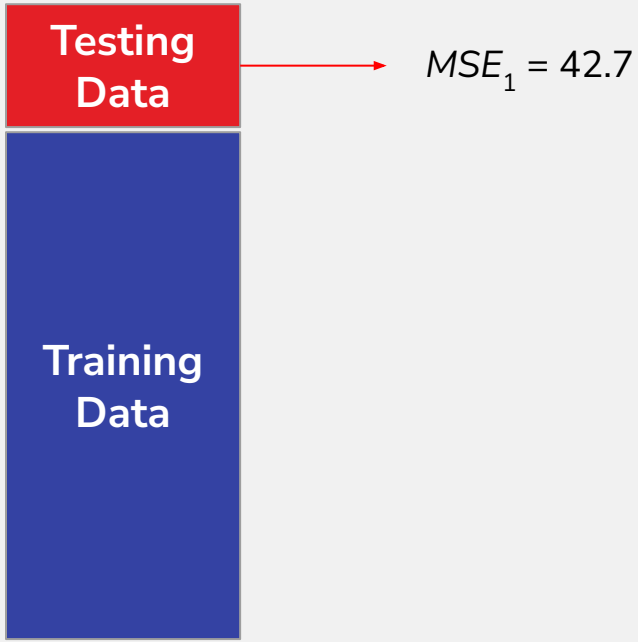
 © 2025 General Assembly

k-Fold Cross-Validation

Next, we mark the first fold as “testing data” and the rest as “training”.

 © 2025 General Assembly

k-Fold Cross-Validation

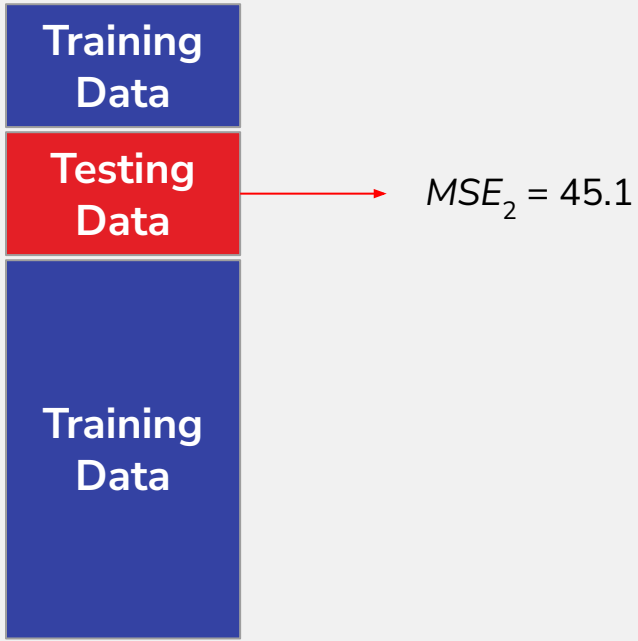


Next, we mark the first fold as “testing data” and the rest as “training”.

We fit a model to the training, and compute some error metric on the testing data. We’ll store this for later.

 © 2025 General Assembly

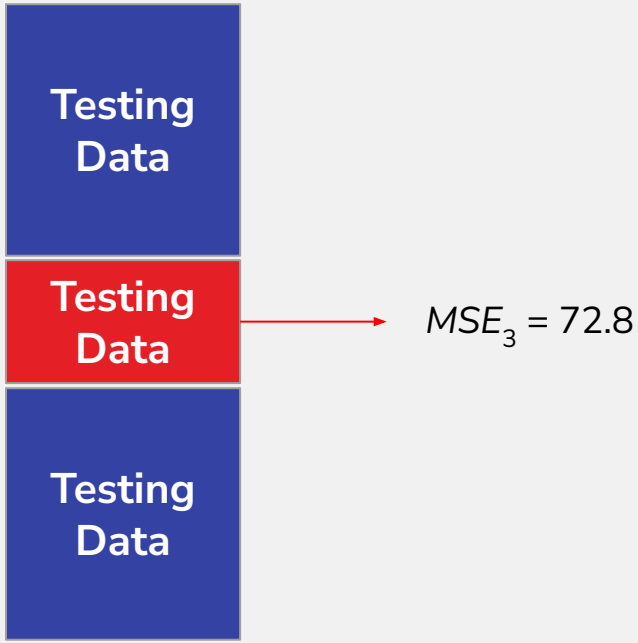
k-Fold Cross-Validation



We repeat this process for the next fold. Fold 2 becomes the testing data, and folds 1, 3, 4, and 5 become training data.

 © 2025 General Assembly

k-Fold Cross-Validation



...and so on.

Sometimes, just due to chance, you'll get an outlying MSE.

k-Fold Cross-Validation

Fold 1	$MSE_1 = 42.7$
Fold 2	$MSE_2 = 45.1$
Fold 3	$MSE_3 = 72.8$
Fold 4	$MSE_4 = 38.2$
Fold 5	$MSE_5 = 49.2$
	$MSE_{CV} = 49.6$

Eventually, we get one error metric per fold.

Finally, we average them together to get a **cross-validated MSE**.

 © 2025 General Assembly

k-Fold Cross-Validation

Fold 1	$MSE_1 = 42.7$
Fold 2	$MSE_2 = 45.1$
Fold 3	$MSE_3 = 72.8$
Fold 4	$MSE_4 = 38.2$
Fold 5	$MSE_5 = 49.2$
	$MSE_{CV} = 49.6$

This value is our final result. This gives us a more robust measurement of model validity. We can compare this to a full-data MSE or a cross validated MSE from a model with different parameters!

 © 2025 General Assembly

k-Folds Cross Validation

Let's see it for ourselves!

- ☐ Open **02-train-test-split-cross-validation.ipynb**

Why not always use this?

ADVANTAGES OF CV

- You get a measure of bias/variance that's more robust to outliers
- You don't "throw away" data - every observation gets to be used as part of model training

 © 2025 General Assembly

Why not always use this?

ADVANTAGES OF CV	DISADVANTAGES OF CV
☐ You get a measure of bias/variance that's more robust to outliers ☐ You don't "throw away" data - every observation gets to be used as part of model training	☐ Since the same data are used to compute the MSE in each fold, by averaging them you're actually introducing some bias into your estimate. (People typically ignore this though). ☐ You need to carry out model fitting *k* times! This is a **huge** disadvantage!!

 © 2025 General Assembly

Other methods

Train-test split and k-folds cross validation are the most common methods.

There are others, such as a special case of cross-validation, **Leave One Out Cross-Validation (LOOCV)**, which is when you let $k = n$.

It's rarely used nowadays, but you might hear about it.

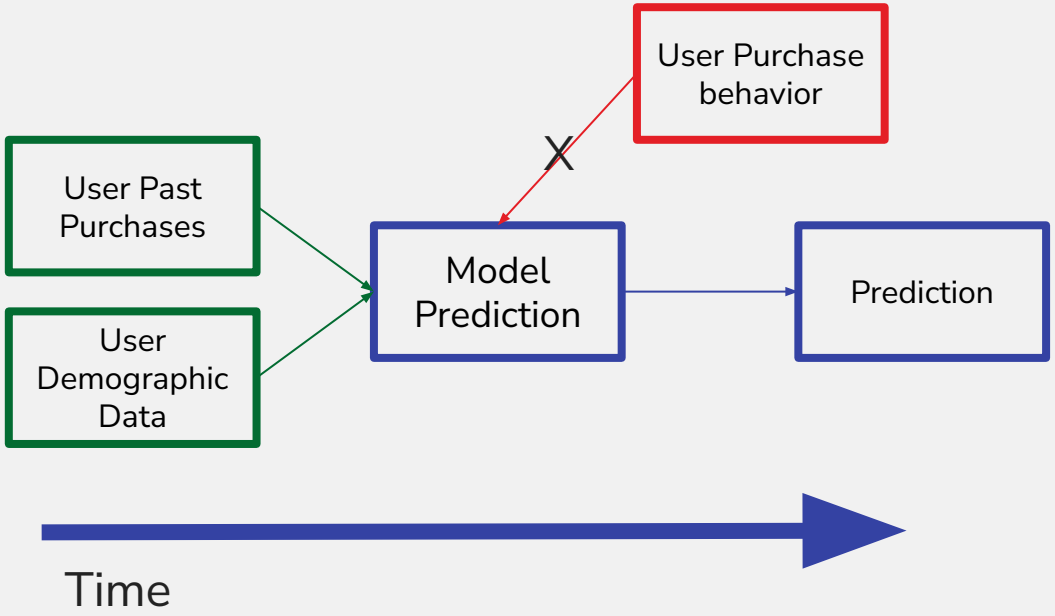
 © 2025 General Assembly

Practical Considerations

Data Leakage

Sometimes data that isn't available at the time of prediction leaks and makes it into the model that is used for prediction.

This is known as **data leakage**.



Data Leakage

Data leakage can occur when:

- ➔ Future information is used to predict the past.
- ➔ Preprocessing, normalization, or splitting errors lead to testing or validation data being used for model training.
- ➔ Validation or process changes can lead to leakage by introducing new information into to the training set.
- ➔ You are working with very large datasets.

 © 2025 General Assembly

Big data

Generally speaking, having more data is a **very** good thing. But large datasets come with some special considerations:

- ➔ Subtle data leakages can be challenging to catch
- ➔ Compute costs and time can grow exponentially
- ➔ Models may overfit spurious correlations that exists in the training data but don't reflect the real world

 © 2025 General Assembly

Production

OK, then what?

Let's say you used train test-split (TTS) or cross-validation (CV) to pick out a model architecture that meets your standards of quality and has a good bias-variance tradeoff.

You're now tasked with deploying this model so that it can be used to make future predictions. **What's your next step?**

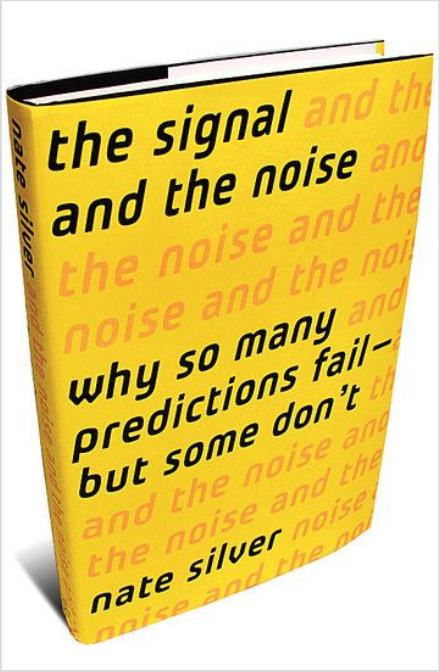
- ☐ What data do you use to fit your final model?

 © 2025 General Assembly

The Signal and the Noise

You can think about ML as modeling the **signal** between X and y and separating it from the **noise**.

If we overfit our model, we're just modeling noise! This happens a lot with inexperienced scientists of any field: including geologists!



Source

Bring It Home

Key takeaways

- It is *always* mathematically possible to get a perfect model on your training data (think about this! How could you do it?).
- Having a good **training MSE** is unimpressive. We need to know our model can generalize to **new data**.
- We can ***holdout*** data from our model using either a train-test split or *k*-fold cross validation.
- Top priority in model prediction is to get a good **testing MSE**, as this measures how your model is ***really*** doing!

 © 2025 General Assembly

GENERAL ASSEMBLY